

block, which runs for a GET request, we make an instance of `EntryForm` with the argument `instance=entry` ❷. This argument tells Django to create the form, prefilled with information from the existing entry object. The user will see their existing data and be able to edit that data.

When processing a POST request, we pass both the `instance=entry` and the `data=request.POST` arguments ❸. These arguments tell Django to create a form instance based on the information associated with the existing entry object, updated with any relevant data from `request.POST`. We then check whether the form is valid; if it is, we call `save()` with no arguments because the entry is already associated with the correct topic ❹. We then redirect to the topic page, where the user should see the updated version of the entry they edited ❺.

If we're showing an initial form for editing the entry or if the submitted form is invalid, we create the context dictionary and render the page using the `edit_entry.html` template.

The `edit_entry` Template

Next, we create an `edit_entry.html` template, which is similar to `new_entry.html`:

```
edit_entry.html {% extends "learning_logs/base.html" %}

{% block content %}

    <p><a href="{% url 'learning_logs:topic' topic.id %}">{{ topic }}</a></p>

    <p>Edit entry:</p>

❶ <form action="{% url 'learning_logs:edit_entry' entry.id %}" method='post'>
    {% csrf_token %}
    {{ form.as_div }}
❷ <button name="submit">Save changes</button>
</form>

{% endblock content %}
```

The `action` argument sends the form back to the `edit_entry()` function for processing ❶. We include the `entry.id` as an argument in the `{% url %}` tag, so the view function can modify the correct entry object. We label the submit button as `Save changes` to remind the user they're saving edits, not creating a new entry ❷.

Linking to the `edit_entry` Page

Now we need to include a link to the `edit_entry` page for each entry on the topic page:

```
topic.html --snip--
    {% for entry in entries %}
        <li>
```